JDBC Application Development on Linux - A Complete Step-by-Step Guide

Role: Senior Java Developer (15+ Years Experience in Java, JDBC, Servlets, JSP & Spring Framework)

This guide explains **everything we did** to develop and execute a **JDBC Application on Linux** from scratch.

Table of Contents

18. Commands Used
19. Conclusion

---

1. What is JDBC?

JDBC stands for **Java Database Connectivity**.

It is a Java API that allows Java applications to communicate with databases such as:

- MySQL
- Oracle
- PostgreSQL
- SQL Server

Using JDBC, Java applications can perform database operations like:

- Creating Tables
- Inserting Records
- Updating Records
- Deleting Records
- Retrieving Records

Without JDBC, Java cannot communicate directly with a database.

Why JDBC?

Almost every Java application stores data in a database.

Examples include:

- Banking Systems
- Hospital Management Systems
- Student Management Systems
- Online Shopping Applications

- Railway Reservation Systems
- Library Management Systems

JDBC acts as a bridge between Java and the database.

---

2. JDBC Architecture

Java Application

|

▼

JDBC API

(Connection, Statement,

PreparedStatement,

ResultSet)

|

▼

MySQL Connector/J

(JDBC Driver)

|

▼

MySQL Database

---

3. Software Required

Software	**Purpose**
Java JDK	Compile and run Java programs
MySQL Server	Database Server
MySQL Connector/J	JDBC Driver

Linux Terminal Execute commands
Text Editor (Nano/VS Code) Write Java Code

---

4. Install Java (JDK)

Update packages. sudo

apt update

Install Java.

sudo apt install default-jdk

Check Java version.

java -version Check

Java Compiler.

javac -version

Example Output:

openjdk 21 javac

21

---

5. Install MySQL Server

Install MySQL.

sudo apt install mysql-server

---

6. Start MySQL Server

Start MySQL.

sudo systemctl start mysql

Enable automatic startup.

sudo systemctl enable mysql

Check MySQL status.

sudo systemctl status mysql

Login.

sudo mysql

---

7. Create Database Inside

MySQL.

Show databases. SHOW

DATABASES;

Create database. CREATE

DATABASE college;

Use database.

USE college;

---

8. Create MySQL User

Linux MySQL usually doesn't allow JDBC to connect using root, so we created a separate user.

Create user.

CREATE USER 'student'@'localhost'

IDENTIFIED BY 'jdbc123'; Grant

privileges.

GRANT ALL PRIVILEGES

ON college.*

TO 'student'@'localhost';

Apply changes. FLUSH

PRIVILEGES;

Exit.

EXIT;

---

9. Install MySQL Connector/J

We installed the JDBC driver. sudo dpkg -i mysql-connector-

j_9.7.0-1ubuntu25.10_all.deb

Verify installation.

dpkg -L mysql-connector-j | grep ".jar"

Output:

/usr/share/java/mysql-connector-j-9.7.0.jar

/usr/share/java/mysql-connector-java-9.7.0.jar

---

10. Create Project Folder mkdir

~/Downloads/JDBCproject Go

inside.

cd ~/Downloads/JDBCproject

---

11. Create Java File Create

the program.

nano StudentManagement.java

Paste the Java code.

Save.

Ctrl + O

Enter

Ctrl + X

---

12. Database Connection Code

Class.forName("com.mysql.cj.jdbc.Driver");

Connection con =

DriverManager.getConnection(

"jdbc:mysql://localhost:3306/college",

"student", "jdbc123");

Explanation:

- Loads JDBC Driver
- Connects Java to MySQL Database

---

13. Create Table

SQL Query.

CREATE TABLE student(id

INT PRIMARY KEY, name

VARCHAR(50),

department VARCHAR(50)

);

Java executes it using:

Statement st = con.createStatement();

st.executeUpdate(query);

---

14. JDBC program

import java.sql.*; import

```
java.util.Scanner;  public
class
StudentManagement {


   // Database Details     static final String URL =
"jdbc:mysql://localhost:3306/college";     static final String USER
= "student";     static final String PASSWORD = "jdbc123";


   static Scanner sc = new Scanner(System.in);


   public static void main(String[] args) {
       try
{
          // Load JDBC Driver
          Class.forName("com.mysql.cj.jdbc.Driver");


          // Create Connection
          Connection con = DriverManager.getConnection(URL, USER, PASSWORD);


          System.out.println("================================");
          System.out.println("   JDBC Student Management");
          System.out.println("================================");
          System.out.println("Database Connected Successfully!");


          createTable(con);
```

```
        int choice;
         do
{
            System.out.println("\n========== MENU ==========");
            System.out.println("1. Insert Student");
            System.out.println("2. Display Students");
            System.out.println("3. Update Student");
            System.out.println("4. Delete Student");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");

            choice = sc.nextInt();

            switch (choice) {
                case 1:
insertStudent(con);
break;
                case
2:
                  displayStudents(con);
break;
                case
3:
```

```
                updateStudent(con);
break;

            case 4:
                deleteStudent(con);
break;
             case
5:
                System.out.println("Thank You!");
                break;

default:
                System.out.println("Invalid Choice!");
            }

        } while (choice != 5);

        con.close();

    } catch (Exception e) {
        e.printStackTrace();
    }
  }

  // Create Table     static void
createTable(Connection con) {
```

```
        try
{
String
sql =
"CREATE
TABLE IF
NOT
EXISTS
student(
"
                + "id INT PRIMARY KEY,"
                + "name VARCHAR(50),"
                + "department VARCHAR(50))";

          Statement st = con.createStatement();
st.executeUpdate(sql);
          st.close();

       } catch (Exception e) {
          e.printStackTrace();
       }
   }

   // Insert Student     static void
insertStudent(Connection con) {
```

```
        try
{

            System.out.print("Enter ID: ");
int id = sc.nextInt();

            sc.nextLine();

            System.out.print("Enter Name: ");
            String name = sc.nextLine();

            System.out.print("Enter Department: ");
            String dept = sc.nextLine();

            String sql = "INSERT INTO student VALUES(?,?,?)";

            PreparedStatement ps = con.prepareStatement(sql);

            ps.setInt(1,            id);
ps.setString(2,           name);
ps.setString(3, dept);

            int rows = ps.executeUpdate();

            if (rows > 0)
```

```
            System.out.println("Student Inserted Successfully!");

            ps.close();

        } catch (Exception e) {
            System.out.println("Error : " + e.getMessage());
        }
    }

    // Display Students static void
displayStudents(Connection con) {        try {

            String sql = "SELECT * FROM student";

            Statement st = con.createStatement();

            ResultSet rs = st.executeQuery(sql);

            System.out.println("\n----------------------------------------");
            System.out.printf("%-10s %-20s %-15s\n", "ID", "NAME", "DEPARTMENT");
            System.out.println("----------------------------------------");

            while (rs.next()) {

                System.out.printf("%-10d %-20s %-15s\n",
```

```
                    rs.getInt("id"),
rs.getString("name"),
rs.getString("department"));
          }

          rs.close();
st.close();

       } catch (Exception e) {
          e.printStackTrace();
       }
   }

   // Update Student     static void
updateStudent(Connection con) {
       try
{

          System.out.print("Enter Student ID to Update: ");
int id = sc.nextInt();

          sc.nextLine();

          System.out.print("Enter New Name: ");
          String name = sc.nextLine();
```

```
            System.out.print("Enter New Department: ");
            String dept = sc.nextLine();

            String sql = "UPDATE student SET name=?, department=? WHERE id=?";

            PreparedStatement ps = con.prepareStatement(sql);

            ps.setString(1, name);
ps.setString(2,             dept);
ps.setInt(3, id);

            int rows = ps.executeUpdate();

            if (rows > 0)
               System.out.println("Record Updated Successfully!");
            else
               System.out.println("Student Not Found!");

            ps.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

```
   // Delete Student     static void
deleteStudent(Connection con) {
        try
{

          System.out.print("Enter Student ID to Delete: ");
int id = sc.nextInt();

          String sql = "DELETE FROM student WHERE id=?";

          PreparedStatement ps = con.prepareStatement(sql);            ps.setInt(1, id);

          int rows = ps.executeUpdate();

          if (rows > 0)
             System.out.println("Record Deleted Successfully!");
          else
             System.out.println("Student Not Found!");

          ps.close();

       } catch (Exception e) {
          e.printStackTrace();
       }
```

```
    }
}
```

18. Compile Program

Compile.

javac -cp .:/usr/share/java/mysql-connector-j-9.7.0.jar StudentManagement.java

Explanation:

- javac → Java Compiler
- -cp → Classpath
- . → Current Directory
- /usr/share/java/mysql-connector-j-9.7.0.jar → JDBC Driver

---

19. Run Program java -cp .:/usr/share/java/mysql-connector-j-9.7.0.jar

StudentManagement

Explanation:

- java → JVM
- -cp → Classpath
- Includes JDBC Driver
- Executes StudentManagement

---

20. Program Output

```
================================
 JDBC Student Management
================================


Database Connected Successfully!
```

========== MENU ==========

1. Insert Student

2. Display Students

3. Update Student

4. Delete Student

5. Exit

Insert Operation

Choice : 1

Enter ID : 101

Enter Name : Abdul

Enter Department : CSE

Student Inserted Successfully!

Display Operation

Choice : 2

ID NAME DEPARTMENT

101 Abdul CSE

Update Operation

Choice : 3

Enter ID : 101

Enter New Name : Abdul Khadeer

Enter Department : AI

Record Updated Successfully!

Delete Operation

Choice : 4

Enter ID : 101

Record Deleted Successfully!

Exit

Choice : 5

Thank You

JDBC Classes Used

Class	Purpose
DriverManager	Creates database connection
Connection	Represents the database connection
Statement	Executes static SQL queries
PreparedStatement	Executes parameterized SQL queries safely
ResultSet	Stores data returned from SELECT queries
Scanner	Reads user input

SQL Commands Used

```
CREATE DATABASE college;
USE college;
CREATE TABLE student( id
INT  PRIMARY  KEY,  name
VARCHAR(50),
department
VARCHAR(50)
);
INSERT INTO student VALUES(?,?,?);
SELECT * FROM student;
UPDATE student SET
name=?,
department=?
WHERE id=?;
DELETE FROM student
```

WHERE id=?;

Linux Commands Used sudo apt update sudo apt install default-jdk sudo apt install mysql-server sudo systemctl start mysql sudo systemctl enable mysql sudo mysql mkdir ~/Downloads/JDBCproject cd ~/Downloads/JDBCproject nano StudentManagement.java javac -cp .:/usr/share/java/mysql-connector-j-9.7.0.jar StudentManagement.java java -cp .:/usr/share/java/mysql-connector-j-9.7.0.jar StudentManagement

---

Project Structure

JDBCproject/

|

├— StudentManagement.java ├—

StudentManagement.class

JDBC Driver Location:

/usr/share/java/mysql-connector-j-9.7.0.jar

What You Learned

By completing this project, you learned to:

- Understand what JDBC is and why it is used.
- Understand JDBC Architecture.
- Install Java (JDK) on Linux.
- Install and configure MySQL Server.
- Create a database and a JDBC user.
- Install MySQL Connector/J.

- Connect a Java application to MySQL.
- Create tables programmatically.
- Perform CRUD operations (Create, Read, Update, Delete).
- Use Connection, Statement, PreparedStatement, and ResultSet.
- Compile and run a Java application with an external JAR file on Linux.

Next Learning Path

Now that you have completed JDBC, the recommended order is:

1. JDBC Advanced Topics (Transactions, Batch Processing, Metadata)
2. Servlets
3. JSP
4. MVC Architecture
5. JSTL and Expression Language (EL)
6. Session Management
7. Filters and Listeners
8. Hibernate
9. Spring JDBC
10. Spring Boot with Spring Data JPA

This progression mirrors how enterprise Java web applications